

PERTEMUAN 6 - QUEUE & DEQUE

LAPORAN PRAKTIKUM STRUKTUR DATA

PERTEMUAN 6 - QUEUE DAN DEQUE

Disusun Oleh:

- Nama: Nadine Valia Azzahra
- NIM: 2C2230007
- PRODI: Sains Data

TUJUAN PRAKTIKUM

1. Memahami konsep Queue dan Deque dalam struktur data.
2. Mengimplementasikan operasi enqueue dan dequeue menggunakan Python.
3. Memahami penerapan Queue pada sistem antrian.
4. Membandingkan efisiensi Queue berbasis list dan deque.

DASAR TEORI

Queue merupakan struktur data linear yang menerapkan prinsip FIFO (First In First Out), yaitu elemen yang pertama masuk akan menjadi elemen pertama yang keluar. Operasi utama pada queue adalah enqueue untuk menambahkan data dan dequeue untuk menghapus data dari depan antrian. Python menyediakan modul `collections.deque` yang dirancang khusus untuk operasi antrian sehingga lebih efisien dibandingkan list biasa. Deque (Double Ended Queue) memungkinkan penambahan dan penghapusan elemen dari kedua sisi dengan kompleksitas waktu yang lebih baik.

LEVEL MUDAH

```
# TUGAS 1 - Queue Sederhana Menggunakan List
```

```
queue = []
```

```
queue.append("Andi")
queue.append("Budi")
queue.append("Citra")
queue.append("Dina")
print(f" Isi Queue : {queue}")
print(f" Jumlah      : {len(queue)} elemen")
```

```
Isi Queue : ['Andi', 'Budi', 'Citra', 'Dina']
Jumlah      : 4 elemen
```

✓ Analisis:

Program membuat queue menggunakan list dan menambahkan empat data yaitu Andi, Budi, Citra, dan Dina. Hasil output menunjukkan seluruh data berhasil disimpan dalam urutan yang sama seperti saat dimasukkan. Program juga menghitung jumlah elemen sehingga pengguna dapat mengetahui ukuran queue saat ini.

```
# TUGAS 2 - Operasi Enqueue dan Dequeue
```

```
queue = []
```

```
for item in ["Produk A", "Produk B", "Produk C"]:
    queue.append(item)
    print(f" Enqueue('{item}') => {queue}")
```

```
keluar = queue.pop(0)
print(f" Dequeue() => Keluar: '{keluar}', Sisa: {queue}")
keluar = queue.pop(0)
print(f" Dequeue() => Keluar: '{keluar}', isa: {queue}")
```

```
Enqueue('Produk A') => ['Produk A']
Enqueue('Produk B') => ['Produk A', 'Produk B']
Enqueue('Produk C') => ['Produk A', 'Produk B', 'Produk C']
Dequeue() => Keluar: 'Produk A', Sisa: ['Produk B', 'Produk C']
Dequeue() => Keluar: 'Produk B', isa: ['Produk C']
```

✓ Analisis:

Program memperlihatkan proses penambahan data menggunakan enqueue dan penghapusan data menggunakan dequeue. Data yang pertama masuk yaitu Produk A menjadi data pertama yang keluar. Hasil ini membuktikan bahwa queue bekerja sesuai prinsip FIFO.

```
# TUGAS 3 - Tampilkan Elemen Front dan Rear
```

```
queue = ["Alpha", "Beta", "Gamma", "Delta"]
```

```
print(f" Queue : {queue}")
print(f" Front : {queue[0]}")
print(f" Rear  : {queue[-1]}")
```

```
Queue : ['Alpha', 'Beta', 'Gamma', 'Delta']
Front  : Alpha
Rear   : Delta
```

Analisis:

Program menampilkan elemen paling depan dan paling belakang pada queue. Front menunjukkan data yang akan diproses terlebih dahulu sedangkan rear menunjukkan data terakhir yang masuk ke dalam antrian.

✓ LEVEL MENENGAH

```
# TUGAS 4 - Menu Queue Interaktif

from collections import deque
class QueueInteraktif:
    def __init__(self):
        self._data = deque()

    def enqueue(self, nilai):
        self._data.append(nilai)
        print(f" [ENQUEUE] '{nilai}' masuk => {list(self._data)}")

    def dequeue(self):
        if self.is_empty():
            print(" [DEQUEUE] Queue kosong!")
            return None
        keluar = self._data.popleft()
        print(f" [DEQUEUE] '{keluar}' keluar => {list(self._data)}")
        return keluar

    def front(self):
        if self.is_empty():
            print(" [FRONT] Queue kosong!")
            return None
        print(f" [FRONT]      : {self._data[0]}")
        return self._data[0]

    def rear(self):
        if self.is_empty():
            print(" [REAR] Queue kosong!")
            return None
        print(f" [REAR]       : {self._data[-1]}")
```

```

        return self._data[-1]

    def is_empty(self):
        return len(self._data) == 0

    def size(self):
        print(f" [SIZE]      : {len(self._data)} elemen")
        return len(self._data)

    def tampilkan(self):
        print(f" [QUEUE]     : {list(self._data)}")

qi = QueueInteraktif()
qi.enqueue("Laporan.pdf")
qi.enqueue("Tugas.docx")
qi.enqueue("Presentasi.pptx")
qi.enqueue("Data.xlsx")
qi.tampilkan()
qi.front()
qi.rear()
qi.size()
qi.dequeue()
qi.dequeue()
qi.tampilkan()

[ENQUEUE] 'Laporan.pdf' masuk => ['Laporan.pdf']
[ENQUEUE] 'Tugas.docx' masuk => ['Laporan.pdf', 'Tugas.docx']
[ENQUEUE] 'Presentasi.pptx' masuk => ['Laporan.pdf', 'Tugas.docx', 'Presentasi.pptx']
[ENQUEUE] 'Data.xlsx' masuk => ['Laporan.pdf', 'Tugas.docx', 'Presentasi.pptx', 'Data.xlsx']
[QUEUE]    : ['Laporan.pdf', 'Tugas.docx', 'Presentasi.pptx', 'Data.xlsx']
[FRONT]    : Laporan.pdf
[REAR]     : Data.xlsx
[SIZE]     : 4 elemen
[DEQUEUE] 'Laporan.pdf' keluar => ['Tugas.docx', 'Presentasi.pptx', 'Data.xlsx']
[DEQUEUE] 'Tugas.docx' keluar => ['Presentasi.pptx', 'Data.xlsx']
[QUEUE]    : ['Presentasi.pptx', 'Data.xlsx']

```

✓ Analisis:

Program mengimplementasikan queue dalam bentuk class sehingga operasi enqueue, dequeue, front, rear, size, dan pengecekan queue kosong dapat dilakukan dengan lebih terstruktur. Penggunaan deque membuat proses penghapusan dari depan menjadi lebih efisien dibandingkan list.

```

# TUGAS 5 - Simulasi Antrian Kasir Minimarket

from collections import deque
class KasirMinimarket:

```

```

def __init__(self, nama_kasir):
    self.kasir = nama_kasir
    self.antrian = deque()
    self.nomor = 1

def masuk_antrian(self, nama):
    no = f"#{self.nomor:03d}"
    self.antrian.append({"no": no, "nama": nama})
    print(f"  Masuk antrian {no}: {nama}")
    self.nomor += 1

def layani(self):
    if not self.antrian:
        print("  Tidak ada pelanggan dalam antrian.")
        return

    p = self.antrian.popleft()
    print(f"  Kasir {self.kasir} melayani {p['no']}: {p['nama']} => SELESAI")

def tampilkan(self):
    if not self.antrian:
        print("  Antrian kosong.")
        return

    print(f"  Antrian Kasir {self.kasir} ({len(self.antrian)} orang):")
    for p in self.antrian:
        print(f"    {p['no']} - {p['nama']}")

kasir = KasirMinimarket("Kasir 1")

kasir.masuk_antrian("Rini")
kasir.masuk_antrian("Dodi")
kasir.masuk_antrian("Sari")
kasir.masuk_antrian("Bayu")

kasir.tampilkan()

kasir.layani()
kasir.layani()

kasir.tampilkan()

```

```

Masuk antrian #001: Rini
Masuk antrian #002: Dodi
Masuk antrian #003: Sari
Masuk antrian #004: Bayu
Antrian Kasir Kasir 1 (4 orang):
#001 - Rini
#002 - Dodi
#003 - Sari
#004 - Bayu
Kasir Kasir 1 melayani #001: Rini => SELESAI

```

```
Kasir Kasir 1 melayani #002: Dodi => SELESAI
Antrian Kasir Kasir 1 (2 orang):
#003 - Sari
#004 - Bayu
```

✓ Analisis:

Program mensimulasikan kondisi nyata pada antrian kasir. Setiap pelanggan memperoleh nomor antrian secara otomatis dan dilayani berdasarkan urutan kedatangan. Hasil menunjukkan pelanggan Rini dan Dodi dilayani lebih dahulu dibanding pelanggan yang datang setelahnya.

```
# TUGAS 6 - Queue Menggunakan Deque

from collections import deque

queue = deque()

for item in ["Node A", "Node B", "Node C", "Node D"]:
    queue.append(item)
    print(f" append('{item}')    => {list(queue)}")

print()

while queue:
    keluar = queue.popleft()
    print(f" popleft()          => Keluar: '{keluar}' | Sisa: {list(queue)}")

append('Node A')    => ['Node A']
append('Node B')    => ['Node A', 'Node B']
append('Node C')    => ['Node A', 'Node B', 'Node C']
append('Node D')    => ['Node A', 'Node B', 'Node C', 'Node D']

popleft()           => Keluar: 'Node A' | Sisa: ['Node B', 'Node C', 'Node D']
popleft()           => Keluar: 'Node B' | Sisa: ['Node C', 'Node D']
popleft()           => Keluar: 'Node C' | Sisa: ['Node D']
popleft()           => Keluar: 'Node D' | Sisa: []
```

✓ Analisis:

Program menunjukkan implementasi queue menggunakan deque dari modul collections. Operasi append digunakan untuk menambah data dan popleft digunakan untuk mengeluarkan data. Seluruh elemen keluar sesuai urutan masuk sehingga karakteristik FIFO tetap terjaga.

TUGAS 7 - Perbandingan Queue List vs Deque

```

import time

N = 50_000

# Dengan List
list_queue = []
t0 = time.perf_counter()
for i in range(N):
    list_queue.append(i)
for i in range(N):
    list_queue.pop(0)
t1 = time.perf_counter()
waktu_list = t1 - t0

# Dengan Deque
deque_queue = deque()
t2 = time.perf_counter()
for i in range(N):
    deque_queue.append(i)
for i in range(N):
    deque_queue.popleft()
t3 = time.perf_counter()
waktu_deque = t3 - t2

print(f"  Jumlah elemen uji   : {N:,}")
print(f"  Waktu List              : {waktu_list:.4f} detik   | Dequeue => O(n) tiap p")
print(f"  Waktu Deque             : {waktu_deque:.4f} detik   | Dequeue => O(1) tiap p")
print(f"  Deque lebih cepat      : {waktu_list / waktu_deque:.1f}x")
print()
print(f"  {'Aspek':<25} {'List':<20} {'Deque'}")
print(f"  {'-'*25} {'-'*20} {'-'*20}")
banding = [
    ("Enqueue (append)", "O(1)", "O(1)"),
    ("Dequeue (pop/left)", "O(n)", "O(1)"),
    ("Akses Front", "O(1)", "O(1)"),
    ("Akses Rear", "O(1)", "O(1)"),
    ("Cocok untuk Queue", "Tidak Efisien", "Sangat Efisien"),
]
for b in banding:
    print(f"  {b[0]:<25} {b[1]:<20} {b[2]}")

```

```

Jumlah elemen uji   : 50,000
Waktu List          : 0.2381 detik   | Dequeue => O(n) tiap pop(0)
Waktu Deque         : 0.0176 detik   | Dequeue => O(1) tiap popleft()
Deque lebih cepat   : 13.5x

```

Aspek	List	Deque
Enqueue (append)	O(1)	O(1)
Dequeue (pop/left)	O(n)	O(1)

Akses Front	$O(1)$	$O(1)$
Akses Rear	$O(1)$	$O(1)$
Cocok untuk Queue	Tidak Efisien	Sangat Efisien

Analisis:

Program melakukan pengujian performa antara list dan deque dengan 50.000 elemen. Hasil menunjukkan deque memiliki waktu eksekusi yang jauh lebih cepat dibandingkan list karena operasi popleft memiliki kompleksitas $O(1)$, sedangkan pop(0) pada list memiliki kompleksitas $O(n)$. Oleh karena itu deque lebih direkomendasikan untuk implementasi queue dalam aplikasi nyata.

KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa Queue merupakan struktur data yang menerapkan konsep FIFO sehingga sangat cocok digunakan untuk sistem antrian. Python menyediakan dua cara implementasi queue yaitu menggunakan list dan deque. Meskipun keduanya dapat digunakan, deque memiliki performa yang lebih baik terutama pada operasi penghapusan data dari depan. Melalui berbagai studi kasus yang dikerjakan, konsep queue dapat diterapkan pada banyak permasalahan nyata seperti antrian kasir, pengelolaan dokumen, dan pemrosesan data secara berurutan.

✓ DISKUSI

1. Apa yang dimaksud dengan Queue?

Struktur data linier yang menyimpan elemen secara berurutan, di mana penambahan data dilakukan di satu ujung (belakang) dan penghapusan data dilakukan di ujung lainnya (depan).

2. Mengapa Queue menggunakan prinsip FIFO?

Karena Queue mensimulasikan antrean di dunia nyata. FIFO (First In, First Out) memastikan elemen yang pertama kali masuk adalah yang pertama kali keluar demi menjaga keadilan urutan proses.

3. Apa perbedaan Enqueue dan Dequeue?

Enqueue: Proses memasukkan atau menambah elemen baru ke dalam antrean (di posisi belakang).

Dequeue: Proses mengeluarkan atau menghapus elemen dari antrean (di posisi depan).

4. Apa perbedaan Queue dan Stack?

Queue: Menggunakan prinsip FIFO (masuk pertama, keluar pertama).

Stack: Menggunakan prinsip LIFO (Last In, First Out - masuk terakhir, keluar pertama).

5. Apa kelebihan Deque dibanding Queue?

Deque (Double-ended Queue) lebih fleksibel karena memungkinkan operasi penambahan dan penghapusan elemen dilakukan dari kedua ujung (depan maupun belakang), sedangkan Queue biasa hanya bisa dari satu arah.

6. Mengapa deque lebih efisien dibanding List untuk Queue?

Menghapus elemen di bagian depan biasa pada List memerlukan pergeseran semua elemen lain (kompleksitas $O(n)$). Sementara Deque dirancang untuk menambah/menghapus di kedua ujung secara langsung dengan kompleksitas konstan ($O(1)$).

7. Sebutkan penerapan Queue dalam sistem operasi

Penjadwalan proses CPU (CPU Scheduling).Antrean pengelolaan memori (Buffer/Cache).Antrean pencetakan dokumen (Print Spooling).

8. Bagaimana Queue digunakan pada layanan perbankan?

Digunakan untuk mengatur antrean nasabah di teller atau customer service. Nasabah yang datang dan mengambil nomor antrean pertama akan dilayani terlebih dahulu sesuai urutan kedatangan.

9. Apa yang dimaksud dengan Process Scheduling?

Mekanisme sistem operasi untuk mengatur dan menjadwalkan proses mana yang berhak menggunakan CPU pada waktu tertentu agar alokasi sumber daya berjalan adil dan optimal.

10. Mengapa Queue penting dalam sistem informasi modern?

Karena Queue membantu menangani lalu lintas data yang tinggi secara asinkron, mencegah sistem kolaps saat terjadi lonjakan permintaan (misal: sistem ticketing, pengiriman pesan massal, atau microservices), dan memastikan pemrosesan data tetap teratur.

